C# Interview Preparation

Top Questions & Answers for .NET Developers

1. C# Basics & Language Fundamentals

Q: What is the difference between value types and reference types?

Value types (int, double, struct, enum) store data directly on the stack. Reference types (class, string, array, delegate) store a reference on the stack pointing to data on the heap. Assigning a value type copies the value; assigning a reference type copies the reference.

```
int a = 5; int b = a; b = 10; // a is still 5 string s1 = "Hello"; string s2 = s1; s2 = "World"; // s1 still "Hello"
```

Q: What is boxing and unboxing?

Boxing converts a value type to object (or an interface type), storing it on the heap. Unboxing extracts the value type back. Both have a performance cost and should be avoided in hot paths.

```
int x = 42; object boxed = x; // boxing int y = (int)boxed; // unboxing
```

Q: What is the difference between 'const' and 'readonly'?

const: compile-time constant, implicitly static, only for primitives/strings. readonly: runtime constant, can be set in constructor, works with any type including objects.

Q: What are nullable types? What is the '??' operator?

Nullable types allow value types to be null (int?, bool?). The ?? (null-coalescing) operator returns the left operand if not null, otherwise the right.

```
int? age = null; int result = age ?? 0; // result = 0 string name = null; string display = name ?? "Unknown";
```

Q: What is the difference between '==' and '.Equals()'?

For value types, both compare values. For reference types, == compares references by default (unless overloaded), while .Equals() compares content. string overloads both to compare content.

2. Object-Oriented Programming (OOP)

Q: What are the four pillars of OOP?

Encapsulation: hiding internal state via access modifiers. Inheritance: a class derives from another (single inheritance in C#). Polymorphism: methods behave differently based on object type (override/virtual). Abstraction: hiding complexity behind interfaces/abstract classes.

Q: What is the difference between abstract class and interface?

Abstract class: can have implementation, fields, constructors; a class can inherit only one. Interface: only method signatures (C# 8+ allows default implementations); a class can implement many. Use abstract class for shared behavior, interface for contracts.

Q: What is method overriding vs method overloading?

Overriding: subclass provides its own implementation of a base class virtual method (same signature, uses override keyword). Overloading: multiple methods with the same name but different parameter lists in the same class.

```
// Override public virtual void Speak() { } public override void Speak() {
Console.WriteLine("Woof"); } // Overload public int Add(int a, int b) => a + b;
public double Add(double a, double b) => a + b;
```

Q: What is the 'sealed' keyword?

Prevents a class from being inherited, or prevents a method from being overridden further in derived classes. Useful for security and performance (JIT can optimize sealed methods).

Q: What is the difference between 'is' and 'as'?

'is' checks if an object is of a given type (returns bool). 'as' attempts to cast and returns null if it fails (no exception). Use 'as' when failure is expected; use direct cast when you expect it to always succeed.

```
if (obj is Dog d) { d.Bark(); } // pattern matching var dog = obj as Dog; // null
if not Dog
```

3. Delegates, Events & Lambda

Q: What is a delegate?

A delegate is a type-safe function pointer. It holds a reference to a method with a matching signature and can be passed around and invoked.

```
delegate int MathOp(int x, int y); MathOp add = (a, b) => a + b;
Console.WriteLine(add(3, 4)); // 7
```

Q: What is the difference between Action, Func, and Predicate?

Action: delegate that takes parameters and returns void. Func: takes parameters and returns a value. Predicate: takes one parameter and returns bool (equivalent to Func).

```
Action print = s => Console.WriteLine(s); Func add = (a, b) => a + b; Predicate
isEven = n => n % 2 == 0;
```

Q: What is an event? How does it differ from a delegate?

An event is a delegate wrapper with restricted access: outside the declaring class you can only subscribe (+=) or unsubscribe (-=), not invoke directly. Events follow the publisher-subscriber pattern.

Q: What is a lambda expression?

An anonymous function written inline using the => syntax. Used with delegates, LINQ, and functional patterns.

```
List nums = new List {1, 2, 3, 4}; var evens = nums.Where(n => n % 2 ==
0).ToList();
```

4. Generics & Collections

Q: What are generics and why use them?

Generics allow writing type-safe, reusable code without specifying the actual type until use. They eliminate the need for boxing/unboxing and reduce runtime errors.

```
public T GetMax(T a, T b) where T : IComparable => a.CompareTo(b) > 0 ? a : b;
```

Q: What is the difference between List, Array, ArrayList?

Array: fixed size, type-safe, fastest access. List: dynamic size, type-safe (generic). ArrayList: dynamic size but stores objects (no type safety, boxing overhead). Always prefer List over ArrayList.

Q: What are generic constraints (where)?

Constraints restrict the types that can be used as type arguments: where T : class (reference type), where T : struct (value type), where T : new() (parameterless constructor), where T : IComparable (implements interface).

Q: What is Dictionary? What is its lookup complexity?

A hash table-based collection of key-value pairs. Lookup, insert, and delete are O(1) on average. Keys must be unique and not null. Use ContainsKey() before accessing a key, or use TryGetValue() for safe access.

5. LINQ (Language Integrated Query)

Q: What is LINQ and what are its two syntaxes?

LINQ provides a unified query syntax for collections, XML, databases, etc. Query syntax (SQL-like) and method syntax (fluent/lambda) are equivalent; both compile to the same IL.

```
// Query syntax var q1 = from n in nums where n > 2 select n * n; // Method syntax
var q2 = nums.Where(n => n > 2).Select(n => n * n);
```

Q: What is deferred execution in LINQ?

LINQ queries are not executed when defined — they execute when enumerated (foreach, ToList(), Count(), etc.). This means the data source is queried fresh each time. Call .ToList() to force immediate execution and cache results.

Tip: Calling ToList() or ToArray() materializes the query and avoids multiple re-executions.

Q: What is the difference between IEnumerable and IQueryable?

IEnumerable: in-memory LINQ (LINQ to Objects), filters data after fetching. IQueryable: translates queries to a provider (e.g., SQL via Entity Framework), filtering happens at the source. Always use IQueryable with EF for performance.

Q: Name common LINQ methods.

Where (filter), Select (project), OrderBy/OrderByDescending, GroupBy, Join, FirstOrDefault, SingleOrDefault, Any, All, Count, Sum, Average, Min, Max, Distinct, Take, Skip, ToList, ToArray, ToDictionary.

6. Async/Await & Multithreading

Q: What is async/await in C#?

async/await is syntactic sugar for writing asynchronous code that looks synchronous. An async method returns Task or Task. The await keyword suspends the method until the awaited task completes without blocking the thread.

```
public async Task FetchDataAsync(string url) { using var client = new
HttpClient(); return await client.GetStringAsync(url); }
```

Q: What is the difference between Task and Thread?

Thread: OS-level thread, heavyweight. Task: higher-level abstraction over the ThreadPool, lightweight. Tasks support continuation, cancellation, and return values. Prefer Task/async over raw threads for most scenarios.

Q: What is a deadlock and how to avoid it?

A deadlock occurs when two threads wait on each other. In async code, calling .Result or .Wait() on a Task in a synchronization context (e.g., UI thread) causes a deadlock. Avoid by using await throughout, or ConfigureAwait(false) in library code.

Tip: Never block async code with .Result or .Wait() in ASP.NET or UI apps.

Q: What is CancellationToken?

CancellationToken allows cooperative cancellation of async operations. Pass it to async methods and check IsCancellationRequested or call ThrowIfCancellationRequested().

```
var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5)); await DoWorkAsync(cts.Token);
```

7. Exception Handling

Q: What is the difference between 'throw' and 'throw ex'?

'throw' rethrows the original exception preserving the stack trace. 'throw ex' creates a new exception losing the original stack trace. Always use 'throw' in catch blocks when rethrowing.

```
catch (Exception ex) { Log(ex); throw; // correct - preserves stack trace // throw ex; // wrong - resets stack trace }
```

Q: What is a finally block?

Code in finally always executes whether or not an exception occurred. Used for cleanup (closing connections, releasing resources). The 'using' statement is preferred for IDisposable objects.

Q: What is the difference between Exception and ApplicationException?

ApplicationException was intended for user-defined exceptions but is now considered legacy. Best practice: derive custom exceptions directly from Exception (or a more specific exception like InvalidOperationException).

8. Memory Management & IDisposable

Q: How does garbage collection work in C#?

The CLR GC automatically reclaims memory. Objects are organized in generations (0, 1, 2). Gen 0 is collected most frequently. Long-lived objects promote to higher generations. GC runs non-deterministically; do not rely on finalizers for critical cleanup.

Q: What is IDisposable and the 'using' statement?

IDisposable defines a Dispose() method for releasing unmanaged resources. The using statement ensures Dispose() is called even if an exception occurs — it compiles to a try/finally block.

```
using (var conn = new SqlConnection(connStr)) { conn.Open(); // ... use conn } // Dispose() called automatically // C# 8+ simplified: using var conn = new SqlConnection(connStr);
```

Q: What is the difference between Dispose and Finalize?

Dispose: called explicitly (via using or directly) to release resources deterministically. Finalize (~Destructor): called by GC non-deterministically for cleanup when Dispose was not called. Implement the full Dispose pattern when dealing with unmanaged resources.

9. Modern C# Features (C# 7–12)

Q: What are tuples in C#?

Tuples group multiple values without creating a class. Named tuples (C# 7+) are more readable.

```
(string Name, int Age) person = ("Alice", 30); Console.WriteLine(person.Name); // Alice
```

Q: What is pattern matching?

Pattern matching (C# 7+) allows checking type and extracting values in one step. Switch expressions (C# 8+) make it even more concise.

```
object obj = 42; if (obj is int n && n > 0) Console.WriteLine("Positive int"); // Switch expression string Describe(object o) => o switch { int i => $"int: {i}", string s => $"string: {s}", _ => "other" };
```

Q: What are records in C#?

Records (C# 9+) are immutable reference types with value-based equality, auto-generated ToString, Equals, and GetHashCode. Perfect for DTOs and data models.

```
public record Person(string Name, int Age); var p1 = new Person("Bob", 25); var p2 = p1 with { Age = 26 }; // non-destructive mutation
```

Q: What is a nullable reference type (C# 8+)?

When enabled (#nullable enable), reference types are non-nullable by default. Adding ? makes them nullable. This helps catch null reference exceptions at compile time.

```
string name = null; // warning: non-nullable string? name = null; // OK: nullable
```

Q: What are init-only properties?

init (C# 9+) allows properties to be set only during object initialization (constructor or object initializer), but not after — similar to readonly but works with object initializers.

```
public class Point { public int X { get; init; } public int Y { get; init; } } var p = new Point { X = 1, Y = 2 }; // p.X = 5; // ERROR after init
```

10. .NET & ASP.NET Core Essentials

Q: What is dependency injection (DI) in .NET?

DI is a design pattern where dependencies are provided to a class rather than created by it. In ASP.NET Core, the built-in IoC container supports 3 lifetimes: Transient (new per request), Scoped (one per HTTP request), Singleton (one for app lifetime).

```
builder.Services.AddTransient(); builder.Services.AddScoped(); builder.Services.AddSingleton();
```

Q: What is middleware in ASP.NET Core?

Middleware are components in the request pipeline, executed in order. Each can short-circuit or pass to the next. Common middleware: Authentication, Authorization, Routing, CORS, Exception handling.

Q: What is Entity Framework Core?

EF Core is an ORM (Object-Relational Mapper) for .NET. It maps C# classes to database tables and lets you query/modify data using LINQ instead of SQL. Supports Code First, Database First, and Migrations.

Q: What is the difference between .NET Framework and .NET Core/.NET 5+?

.NET Framework: Windows-only, legacy. .NET Core (now .NET 5+): cross-platform (Windows, Linux, macOS), open-source, better performance, actively developed. New projects should always target .NET 8+.

11. Quick Tips & Common Mistakes to Avoid

Avoid var overuse: Use var for obvious types (var list = new List()), but be explicit when the type aids readability.

string vs StringBuilder: Use StringBuilder for repeated string concatenation in loops — strings are immutable, so + creates new objects each time.

Null checks: Use null-conditional ?. and null-coalescing ?? to write safe, concise null checks: user?.Name ?? "Guest".

Prefer interfaces: Code to interfaces (IList vs List) for flexibility and testability.

async all the way: Don't mix sync and async. Await all the way up the call chain to avoid deadlocks.

Dispose unmanaged resources: Always use 'using' for SqlConnection, HttpClient, StreamReader, etc.

Avoid magic numbers: Use constants or enums instead of hard-coded numbers/strings.
